



Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

Proyecto de Carrera: Ingeniería en Informática

Unidad Curricular: Lenguaje y Compiladores

Expresiones regulares, su capacidad de reconocimiento léxico y vinculación con los autómatas.

**Empresa: VISIONLY
Mantenlo simple!**

Profesor:
Felix Márquez

Integrantes:
Ana Santamaria, V-21251440
Carlos Martínez V-27955202
Genesis Moya V-31370339
Naleska Carrera V-31521506

Ciudad Guayana, mayo 14 del 2026.

INTRODUCCIÓN

En el ecosistema de la informática, los lenguajes de programación actúan como el puente fundamental entre el pensamiento lógico humano y la ejecución binaria de las máquinas. Sin embargo, para que este puente sea funcional, se requiere de los **compiladores**: sistemas de software altamente complejos encargados de traducir, optimizar y validar el código fuente. Estos componentes no solo aseguran que las instrucciones sean comprensibles para el hardware, sino que también garantizan la integridad estructural de los programas, permitiendo el desarrollo de software escalable y eficiente que sostiene la tecnología actual.

El estudio de estos sistemas y su procesamiento es una piedra angular de la computación, permitiendo que las ideas abstractas se transformen en instrucciones ejecutables. Este informe explora la estrecha relación entre las **expresiones regulares**, la **teoría de autómatas** y su aplicación práctica fundamental en las primeras etapas de la compilación: el **análisis léxico**. A través del examen de conceptos clave como el Teorema de Kleene y el algoritmo de construcción de Thompson, se analizará cómo patrones algebraicos se convierten en mecanismos lógicos capaces de clasificar el vocabulario de un lenguaje con total precisión.

1. Conceptos y Operaciones de las Expresiones Regulares

Las Expresiones Regulares (ER) son una herramienta tanto matemática como computacional que se emplea para definir y trabajar con lenguajes formales. En el ámbito de la informática, se presentan como series de caracteres que forman un patrón de búsqueda, y son esenciales para el reconocimiento léxico, ya que facilitan a los compiladores y a los analizadores de texto detectar elementos válidos (tokens) como palabras clave, identificadores o números en un código fuente (Aho et al., 2007).

En la teoría de los autómatas, una expresión regular describe un "lenguaje regular" (un grupo de cadenas que se forman a partir de un alfabeto). Dado un alfabeto Σ (un conjunto limitado de símbolos), las expresiones regulares fundamentales se establecen de la siguiente forma (Sipser, 2013):

- Conjunto vacío: Una expresión regular que representa un lenguaje que no tiene ninguna cadena.
- Cadena vacía: Una expresión regular que representa el lenguaje que únicamente incluye la cadena vacía (de longitud cero).
- Para cualquier símbolo del alfabeto, a es una expresión regular que representa el lenguaje que solo contiene la cadena " a ".

Operaciones Básicas:

Cualquier expresión regular compleja se genera utilizando de forma recursiva tres operaciones algebraicas elementales sobre las expresiones fundamentales (Hopcroft et al., 2007). Si R_1 y R_2 son expresiones regulares, se pueden realizar las siguientes acciones:

A. Unión (Alternancia o Disyunción):

Se representa como $R_1 \cup R_2$ (en lenguajes de programación frecuentemente se utiliza el símbolo `|`). Se refiere al lenguaje que incluye las cadenas que corresponden a R_1 , a R_2 , o a ambos.

- **Concepto:** Funciona como un "O" lógico.
- **Ejemplo:** Si la expresión es `a | b`, el analizador léxico identificará la cadena "a" o "b".
- **Aplicación léxica:** Para especificar que un dígito puede ser `0|1|2|3|4|5|6|7|8|9`.

B. Concatenación:

Se simboliza como $R_1 \cdot R_2$ (o simplemente R_1R_2). Indica el lenguaje que se crea al tomar cualquier cadena de R_1 y unirla (conectarla al final) con cualquier cadena de R_2 .

- **Concepto:** Funciona como un "Y" en secuencia (esto seguido de aquello).
- **Ejemplo:** Si R_1 es `a` y R_2 es `b`, la concatenación R_1R_2 resulta en la expresión `ab`, que reconoce únicamente la cadena "ab".
- **Aplicación léxica:** Para formar palabras, como la palabra reservada `if`, que es la unión de `i` y `f`.

C. Cierre de Kleene (Estrella de Kleene):

Se simboliza como R^* **explica** el **idioma** que se **crea** al **unir** cero o más **ejemplos** de cadenas de **R**.

- **Concepto:** Indica repetición (desde ninguna vez hasta el infinito).
- **Ejemplo:** La expresión a^* reconocerá la cadena vacía ϵ , "a", "aa", "aaa", y así sucesivamente.
- **Aplicación léxica:** Sirve para definir identificadores. Por ejemplo, un identificador en programación inicia con una letra y puede ir seguido de cualquier cantidad de letras y números: $\text{letra}(\text{letra} \mid \text{número})^*$.

2. Aplicación de análisis léxico en compiladores

Para comprender mejor es importante resaltar su terminología y contexto histórico.

¿Qué significa "Léxico"?

La palabra "léxico" proviene del griego *lexikós*, que significa "perteneciente o relativo al vocabulario". En la lingüística humana, el léxico es el conjunto de palabras válidas que conforman un idioma. Llevado a las ciencias de la computación, el **léxico de un lenguaje de programación** es el conjunto de símbolos y palabras válidas (su "vocabulario") que el lenguaje puede entender (como `if`, `while`, `+`, `=`, etc.). Por lo tanto, un "análisis léxico" no es más que la lectura y validación de que el texto escrito por el programador pertenece realmente al vocabulario permitido por ese lenguaje, antes de intentar entender qué significa la oración completa.

Origen y Evolución de los Analizadores Léxicos

En los inicios de la informática, durante el desarrollo de los primeros compiladores como el de FORTRAN en la década de 1950 (liderado por John Backus), no existía una

separación clara entre el análisis léxico y el sintáctico. El código que leía los caracteres y el que estructuraba la lógica estaban mezclados en rutinas complejas y difíciles de mantener.

La formalización de esta fase surgió en las décadas de 1960 y 1970, gracias a la unión de la teoría de lenguajes formales (impulsada por lingüistas como Noam Chomsky) y los avances en matemáticas computacionales. El matemático Stephen Kleene introdujo las **expresiones regulares**, que permitieron describir patrones de texto de forma algebraica (Kleene, 1956). Más tarde, en 1975, Mike Lesk y Eric Schmidt (quien luego sería CEO de Google), trabajando en Bell Labs, crearon **Lex**, el primer programa generador automático de analizadores léxicos (Lesk, 1975). Lex estandarizó la práctica de separar el análisis léxico como una fase independiente y modular, transformando expresiones regulares en Autómatas Finitos de forma automática, una arquitectura que sigue siendo el estándar hoy en día.

El Proceso de Análisis Léxico (Scanning)

El análisis léxico (también conocido como *scanning*) es la primera fase del proceso de compilación. Su objetivo principal es leer el código fuente, carácter por carácter, y agruparlos en secuencias lógicas con significado llamadas tokens (Aho et al., 2006).

Para entender su aplicación, es vital dominar tres conceptos fundamentales que interactúan en esta fase:

- **Token:** Es un símbolo abstracto que representa una categoría léxica (ej. un identificador, una palabra reservada, un operador matemático). Es la salida del analizador léxico.
- **Patrón (Pattern):** Es la regla que describe cómo se forma un token. En la práctica, estos patrones se definen utilizando expresiones regulares, las cuales luego son implementadas a través de Autómatas Finitos Deterministas (AFD) para un reconocimiento rápido en memoria (Louden, 1997).

- **Lexema (Lexeme):** Es la secuencia real de caracteres en el código fuente que coincide con el patrón de un token específico.

El analizador léxico actúa como un filtro inicial: descarta los espacios en blanco, los saltos de línea y los comentarios, dejando un flujo de tokens limpio para que la siguiente fase (el analizador sintáctico o *parser*) evalúe la gramática del código.

Ejemplo: De una estructura simple a una categorizada

Código fuente de entrada:

Python

```
if (precio >= 100):
```

Proceso de Tokenización (Salida del Analizador Léxico):

Al pasar esta línea por el analizador, el flujo de caracteres se divide y clasifica de la siguiente manera:

Lexema	Patrón Lógico (Expresión Regular)	Token (Categoría)
if	Coincidencia exacta con la palabra clave	RESERVED_WORD
(	Símbolo de apertura de paréntesis	LEFT_PAREN
precio	<code>\$(a-zA-Z_)[a-zA-Z0-9_]*\$</code>	IDENTIFIER
>=	Operador relacional mayor o igual	RELATIONAL_OP
100	<code>\$(0-9)+\$</code>	NUMBER_LITERAL

)	Símbolo de cierre de paréntesis	RIGHT_PAREN
:	Símbolo de dos puntos	COLON

3. El Teorema de Kleene y su Equivalencia con los Autómatas Finitos

El Teorema de Kleene, propuesto por Stephen Cole Kleene en la década de 1950, es el resultado fundamental de la teoría de autómatas que establece la equivalencia total entre las expresiones regulares (ER) y los autómatas finitos (AF). Para profundizar en la equivalencia entre los distintos tipos de Autómatas Finitos (AF), es fundamental entender que, aunque parezcan diferentes en su funcionamiento interno, todos reconocen la misma familia de lenguajes: los lenguajes regulares.

La equivalencia se divide principalmente en la relación entre los modelos Deterministas, No Deterministas y con transiciones vacías.

$$AFN-\epsilon \equiv AFN \equiv AFD$$

- **AFD (Determinista):** Para cada estado y símbolo, existe exactamente una transición. Es fácil de programar pero difícil de diseñar a mano para patrones complejos.
- **AFN (No Determinista):** Para un símbolo, puede haber múltiples caminos o ninguno. Permite "adivinar" la ruta correcta.
- **AFN- ϵ (Con transiciones vacías):** Permite cambiar de estado sin leer ningún símbolo de la entrada. Es el más flexible para el diseño humano.

Este teorema de Kleene demuestra la clase de lenguajes que pueden ser descritos algebraicamente mediante patrones es idéntica a la clase de lenguajes que pueden ser reconocidos mecánicamente por una máquina de estados.

El teorema postula una equivalencia de tres vías. Un lenguaje L es regular si y solo si puede ser definido por:

- Un Autómata Finito Determinista (AFD).
- Un Autómata Finito No Determinista (AFN).
- Una Expresión Regular (ER).
- Puede ser representado por un grafo de transición

Por lo que: $ER \equiv AFN \equiv AFD$. Si puedes dibujarlo como un autómata, puedes escribirlo como una expresión regular, y viceversa.

Esta equivalencia permite entender los lenguajes regulares desde dos perspectivas: la declarativa (ER), que define "qué" queremos encontrar, y la procedural (AF), que define "cómo" lo reconocemos

Teorema de Síntesis: De ER a Autómatas

Esta parte del teorema garantiza que para toda expresión regular existe un autómata finito equivalente. mediante el uso del algoritmo de Thompson que es el método estándar y recursivo para esta conversión. Construye un AFN- ϵ (autómata no determinista con transiciones vacías) ensamblando bloques básicos:

- **Base:** Crea máquinas simples para el símbolo vacío ($\emptyset$), la cadena vacía (ϵ) y símbolos individuales del alfabeto (a).
- **Inducción:** Aplica reglas para combinar estos bloques mediante los operadores de:
 - **Unión ($R + S$):** Se crea un nuevo inicio que bifurca hacia los autómatas de R y S usando transiciones vacías (ϵ).
 - **Concatenación (RS):** El estado final de R se conecta al inicio de S .
 - **Cierre de Kleene (R^*):** Se añaden bucles de retroalimentación con ϵ para permitir la repetición infinita o el salto (cadena vacía).
- **Algoritmo de Glushkov:** Una alternativa que produce un AFN directamente sin transiciones vacías

Teorema de Análisis: De Autómatas a ER

Demuestra que cualquier lenguaje aceptado por un autómata finito puede ser descrito mediante una expresión regular. Los métodos principales incluyen:

- **Eliminación de Estados:** Se reducen los estados del autómata uno a uno, actualizando las etiquetas de las transiciones restantes con expresiones regulares complejas que preserven los caminos eliminados.
- **Lema de Arden:** Trata al autómata como un sistema de ecuaciones lineales de lenguajes. Utiliza la regla fundamental de que si $X = AX \cup B$ ($\epsilon \in A$), entonces la solución única es $X = A^* B$.
- **Inducción de Caminos-K:** El método original de Kleene que define rutas entre estados basándose inductivamente en el índice del estado intermedio más alto permitido.

Jerarquía de Transformaciones y Equivalencias Internas

La equivalencia de Kleene se extiende a través de diferentes modelos de máquinas para optimizar la ejecución:

AFN $\equiv$ AFD: Todo autómata no determinista puede convertirse en un autómata determinista equivalente mediante el Algoritmo de Construcción de Subconjuntos.

Minimización: Un AFD puede reducirse a su versión más eficiente (con el menor número de estados posible) sin alterar el lenguaje que reconoce, facilitando la comparación entre expresiones regulares.

Aplicaciones Prácticas

El legado de este teorema es la base de la informática moderna:

Análisis Léxico: Compiladores y herramientas como Flex o Lex usan ER para definir tokens y las compilan automáticamente en

AFDs minimizados para escanear código a alta velocidad.

Motores de Búsqueda: Herramientas como grep, awk y sed implementan estos algoritmos para garantizar un rendimiento lineal ($O(n)$) en la búsqueda de patrones complejos.

Verificación Formal: Se utilizan autómatas para modelar y verificar protocolos de red y hardware, asegurando que los sistemas nunca entren en estados de error.

4. Construcciones de Expresiones Regulares a Autómatas ($ER \rightarrow AFN$)

La transformación de Expresiones Regulares (ER) a Autómatas Finitos No Deterministas (AFN), a menudo con transiciones vacías (AFN-), se lleva a cabo principalmente a través del algoritmo de Construcción de Thompson.

El algoritmo de Thompson es un método inductivo que descompone una expresión regular en sus subexpresiones constituyentes. Su propósito principal es facilitar la implementación de analizadores léxicos, permitiendo que patrones complejos descritos mediante álgebra de lenguaje sean convertidos en grafos de estados. Según Hopcroft et al. (2007), una de las mayores ventajas de este algoritmo es que el autómata resultante tiene exactamente un estado inicial y un estado final, lo que simplifica la concatenación de módulos más grandes.

Además, es importante destacar que este enfoque permite la creación de autómatas que pueden ser utilizados para el reconocimiento de patrones en cadenas de texto, lo que resulta fundamental en aplicaciones como la búsqueda de texto y la validación de entradas.

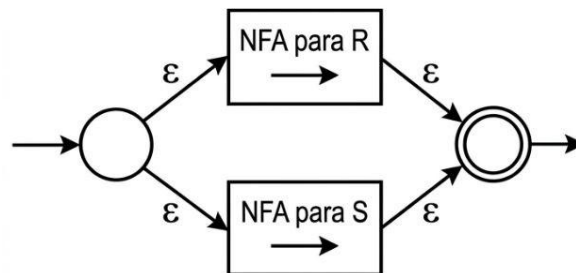
Construcciones Básicas:

El algoritmo es inductivo, por lo tanto construye autómatas complejos a partir de los componentes más simples.

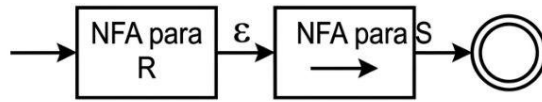
- Símbolo del alfabeto (a): Representa el reconocimiento de un carácter específico del alfabeto. Se grafica como una transición entre dos estados etiquetada con el símbolo en cuestión.
- Símbolo vacío (ϵ): Es un componente técnico que permite al autómata cambiar de estado sin consumir ningún símbolo de la entrada. Es esencial para implementar la lógica de control en estructuras complejas.

Operadores Lógico-Estructurales en la Construcción

A. Unión: El Operador de Elección ($R|S$): La unión representa la capacidad del lenguaje de elegir entre dos rutas posibles. Estructuralmente, el algoritmo crea un nuevo estado inicial que se ramifica hacia los inicios de los autómatas de R y S mediante transiciones ϵ . Al finalizar ambos procesos, estos convergen en un nuevo estado final común. Esta estructura garantiza que el autómata pueda aceptar una cadena si esta es reconocida por cualquiera de las dos subexpresiones.

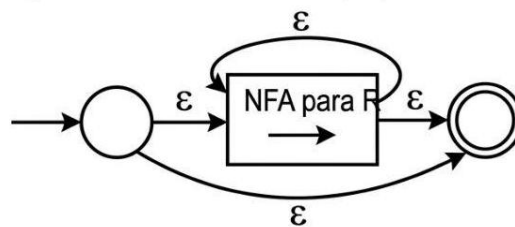


B. Concatenación: El Orden Secuencial (RS): La concatenación modela la aparición de un patrón inmediatamente después de otro. En la construcción de Thompson, esto se logra uniendo el estado final del primer autómata (R) con el estado inicial del segundo (S). Esta conexión puede ser directa (fusionando estados) o mediante una transición ϵ , asegurando que la secuencia de símbolos se valide en el orden estricto definido por la expresión regular.



C. Cerradura de Kleene: El Bucle de Repetición (R^*): La cerradura de Kleene es el operador que permite la recursividad o repetición (cero o más veces). Para su construcción gráfica, se rodea al autómata de la expresión base (R) con cuatro transiciones ϵ estratégicas:

1. Una que permite volver del final al inicio de R (repetición).
2. Una que permite saltar desde el nuevo inicio al nuevo final (representando la cadena vacía o cero repeticiones).
3. Dos que conectan los nuevos estados de control con la estructura interna.

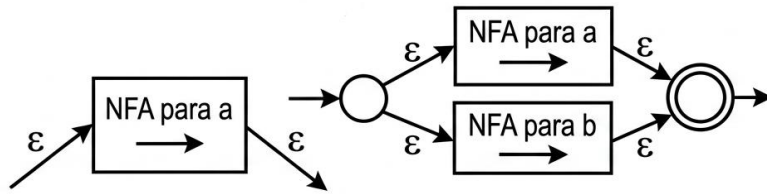


Vinculación con el Análisis Léxico

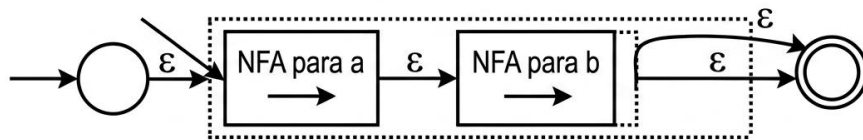
La importancia de estas construcciones gráficas radica en que son el paso previo indispensable para la optimización. Una vez generado el AFN- ϵ mediante Thompson, este suele transformarse en un Autómata Finito Determinista (AFD) mediante el algoritmo de subconjuntos, garantizando que el reconocimiento de tokens durante la fase de compilación se realice en tiempo lineal y de forma eficiente.

Ejemplo de construcción de Thompson: $(a|b)^*c$

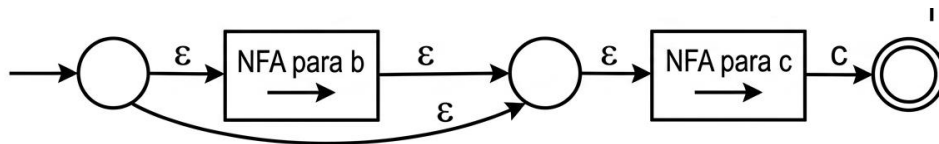
1. Unión: Se crean rutas paralelas para a y b . Un estado inicial se bifurca mediante transiciones ϵ hacia los autómatas individuales, que luego convergen en un estado común.



2. Cerradura de Kleene: Se envuelve la estructura anterior con un bucle de retroalimentación (para repetir "a" o "b") y un camino de salto (para permitir la cadena vacía), ambos mediante transiciones ϵ .



3. Concatenación: El estado final de la cerradura se conecta (ya sea directamente o por ϵ) al estado inicial del autómata que reconoce el símbolo **c**.



Como resultado tenemos un autómata que garantiza que cualquier secuencia de "a" y "b" (incluyendo ninguna) sea procesada antes de finalizar obligatoriamente con el símbolo **c**.

CONCLUSION

En definitiva, la evolución del análisis léxico, desde las rutinas mezcladas de los años 50 hasta la modularidad estandarizada por herramientas como Lex demuestra el poder de la formalización matemática en el desarrollo de software. La equivalencia establecida por el Teorema de Kleene entre expresiones regulares y autómatas finitos no es solo un concepto teórico; es la garantía de que cualquier patrón de búsqueda puede ser reconocido de forma eficiente y lineal por una computadora. Al emplear métodos como la Construcción de Thompson, los desarrolladores pueden diseñar analizadores léxicos robustos que sirven como el primer filtro indispensable en la creación de compiladores, asegurando que solo el contenido válido progrese en el flujo de procesamiento de datos.

Referencias Bibliográficas

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson Addison Wesley.

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson Education.

Barceló, P. (s.f.). *Expresiones regulares (autómatas) con variables y sus aplicaciones*.
Departamento de Ciencias de la Computación, Universidad de Chile.

Bavera, F. P., & Nordio, D. M. (2002). *JTLex: Un generador de analizadores léxicos traductores* [Tesina de grado, Universidad Nacional de Río Cuarto]. Repositorio Digital UNRC.

GeeksforGeeks. (2022, 16 de enero). *Conversion of regular expression to finite automata*.
<https://www.geeksforgeeks.org/theory-of-computation/conversion-of-regular-expression-to-finite-automata/>

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to automata theory, languages, and computation* (3rd ed.). Pearson.

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3.^a ed.). Addison-Wesley.
[https://fliphtml5.com/ejte/lkvc/Teoria de automatas%2C lenguajes y computacion%2C 3ra edicion - Hopcroft%2C Motwani y Ullman/6](https://fliphtml5.com/ejte/lkvc/Teoria_de_automas%2C_lenguajes_y_computacion%2C_3ra_edicion_-_Hopcroft%2C_Motwani_y_Ullman/6)

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3.^a ed.). Pearson Educación.

Kleene, S. C. (1956). Representation of events in nerve nets and finite automata. En C. E.

Shannon & J. McCarthy (Eds.), *Automata studies* (pp. 3–41). Princeton University Press.

Lesk, M. E. (1975). *Lex - A lexical analyzer generator* (Computing Science Technical Report No. 39). Bell Laboratories.

Louden, K. C. (1997). *Compiler construction: Principles and practice*. PWS Publishing Company.

Sipser, M. (2013). *Introduction to the theory of computation* (3rd ed.). Cengage Learning.